REAL-TIME GAME LEADERBOARD —

Student Name: Yomi Odukoya

Student ID: 301907943

GitHub Repository: `https://github.com/okaziecs/Odukoya_Yomi_301907943.git`

1. Executive Summary & Organisational Context

1.1 Selected Organisation Overview

Global Arcade Hub operates an international network of physical arcade gaming kiosks integrated with dynamic online multiplayer titles. The platform serves millions of daily active users across North America, Europe, and Asia-Pacific. The core business infrastructure requires real-time telemetry processing, global leaderboard rankings, microtransaction security, and strict data privacy compliance across international jurisdictions (GDPR & CCPA).

1.2 Polyglot Persistence Architectural Strategy

To meet the operational requirements of high-throughput score telemetry alongside transactional player identity management, a **Polyglot Persistence Architecture** was designed and implemented:

- **Relational Layer (Neon / PostgreSQL 16):** Acts as the immutable source of truth for core identity, player balances, microtransactions, location metadata, and security governance. This layer prioritizes **Consistency and Partition Tolerance (CP)** under the CAP theorem.
- **Document Stream Layer (MongoDB Atlas):** Serves as an horizontally scalable, high-velocity event logger for real-time player score submissions, gameplay telemetry, and dynamic aggregations. This layer prioritizes **Availability and Partition Tolerance (AP)**.

2. Technical Implementation Across Portfolio Modules

2.1 Relational Schema Design & Normalization (Portfolio 1)

The core relational schema was implemented in PostgreSQL under a dedicated schema namespace (`portfolio`). The design satisfies Third Normal Form (3NF) to minimize data redundancy and maintain referential integrity across entities:

- **`player`**: Manages unique user identity, authentication hashes, and region metadata.
- **`game`**: Stores title attributes, game modes, and operational status.
- **`score_entry`**: Captures verified score events with strict foreign keys (`ON DELETE CASCADE`), timestamp defaulting (`CURRENT_TIMESTAMP`), and data domain check constraints (`CHECK (score >= 0)`).

2.2 Indexing & Query Profiling (Portfolio 2)

To optimize query execution time for real-time leaderboards, performance profiling was conducted using `EXPLAIN ANALYZE`:

- **Unindexed Baseline:** Queries retrieving top player rankings per game resulted in costly sequential scans (`Seq Scan`) across all rows.
- **Optimization:** Implemented a targeted composite B-Tree index:
- SQL

```
CREATE INDEX idx_score_game_achieved ON
portfolio.score_entry(game_id, score DESC);
```

- **Performance Impact:** Converted sequential table scans into fast **Index Scans**, reducing execution time from orders of magnitude down to sub-millisecond execution (`~0.048 ms`).

2.3 Transaction Management & Concurrency Control (Portfolio 3)

To eliminate race conditions, phantom reads, and double-spending during concurrent score uploads and balance updates, strict ACID transaction boundaries were built:

- **Explicit Row Locking:** Utilized `SELECT ... FOR UPDATE` to lock specific player rows during processing.
- **Atomicity & Rollbacks:** Wrapped database state changes in `BEGIN; ... COMMIT;` blocks, with error handling to execute a `ROLLBACK;` if check constraints (such as `CHECK (p3_processed_count >= 0)`) are violated.

2.4 NoSQL Event Streaming & Validation (Portfolio 4)

High-frequency game events are logged in MongoDB Atlas using `$jsonSchema` document validation to enforce schema governance at the application boundaries:

- **Document Validation:** Enforced required fields (`playerId`, `gameId`, `score`, `eventType`, `occurredAt`) and strict BSON types.

- **Defensive Error Handling:** Standardized `mongosh` scripts utilizing `try/catch` logic catch duplicate key errors (`Code 11000`) and schema validation failures (`Code 121`), maintaining cluster stability without crashing application worker nodes.

2.5 Security, Governance, & Distributed Cloud Design (Portfolio 5)

- **Role-Based Access Control (RBAC):** Defined distinct roles (`arcade_app_service`, `arcade_analyst`, `arcade_auditor`) adhering to the Principle of Least Privilege.
- **Row-Level Security (RLS) & PII Masking:** Protected player privacy under GDPR by enforcing RLS policies (`player_self_service_policy`) and establishing masked views (`v_masked_player_analytics`) that redact sensitive identity attributes.
- **Audit Triggers:** Deployed automated PL/pgSQL triggers capturing DML state changes in `portfolio.security_audit_log`.
- **Hybrid Cloud Topology:** Formulated a multi-region deployment topology (`Odukoya_Yomi_301907943_P5_CloudConfig.json`) detailing PostgreSQL RDS Multi-AZ instances in `us-east-1` (with asynchronous read replicas in `eu-central-1` and `ap-southeast-1`) combined with a MongoDB Atlas 3-shard cluster using a compound hashed key (`{ eventType: "hashed", occurredAt: 1 }`).

3. Implementation Verification & Logs

3.1 PostgreSQL Query Optimization Verification (`EXPLAIN ANALYZE`)

Plaintext

```
QUERY PLAN
------------------------------------------------------------------------
-------------------------------------------------
Index Scan using idx_score_game_achieved on score_entry
(cost=0.28..8.30 rows=1 width=36) (actual time=0.034..0.036 ms)
  Index Cond: ((game_id = 101) AND (score > 800))
Planning Time: 0.082 ms
Execution Time: 0.048 ms
```

3.2 MongoDB Atlas Telemetry Execution Log

Plaintext

```
global_arcade> db.leaderboard_events.createIndex({ eventType:
"hashed", occurredAt: 1 })
idx_shard_key_telemetry

global_arcade> db.leaderboard_events.stats()
```

```
{
  ns: 'global_arcade.leaderboard_events',
  count: 2,
  storageSize: 36864,
  totalIndexSize: 36864,
  ok: 1
}
```

4. Technical Reflection

Developing the **Global Arcade Hub** database platform provided deep insights into modern database architecture:

1. **Trade-offs in Polyglot Architecture:** The project highlighted that no single database technology solves all modern application challenges. Using PostgreSQL for ACID-compliant transactions guarantees financial accuracy, while MongoDB provides the write-throughput necessary for telemetry. Decoupling these concerns prevents system bottlenecks.
2. **Shift-Left Database Security:** Implementing Row-Level Security (RLS) and dynamic data masking within PostgreSQL proved that security must be treated as a core database concern rather than offloaded solely to backend APIs.
3. **Execution Profiling Matters:** Practical query tuning using `EXPLAIN ANALYZE` demonstrated how unindexed queries degrade exponentially under load, proving the importance of index strategy in cloud environments.

5. References & Generative AI Use Declaration

5.1 References

- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- MongoDB, Inc. (2024). *MongoDB Documentation: Schema Validation & Aggregation Framework*.
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation: Concurrency Control & Row-Level Security*.

5.2 Declaration of Generative AI Assistance

In compliance with university academic guidelines, Generative AI (Gemini) was utilized during this portfolio project as a technical writing and debugging collaborator for:

- Refining SQL schema syntax and MongoDB `mongosh` error-handling structures.
- Structuring README layout formats and verifying GitHub folder naming compliance.
- Reviewing edge-case transactional rollback conditions.

All database DDL/DML scripts, schema normalizations, index profiling, execution steps, and final evaluations were produced, tested, and validated by **Yomi Odukoya**.